

Patient Follow-up Agent - Deployment Evidence

Consolidated evidence pack — seven reports, their raw artifacts, and a re-check of every hash

Prepared for	Clinic operations and IT decision-makers
Purpose	Deployment acceptance: what was measured, on which revision, and what remains unproven
Evidence revision	fbcc6c1ca09a7624fd5f01016e674cacdc4a5287
Pack location	proposal/evidence/
Contents	7 numbered reports, 3 raw artifacts, 4 visual artifacts
Document date	26 September 2026
Rendered by	proposal/scripts/build_deployment_evidence_pdf.py

1. How to read this pack

The seven reports under `proposal/evidence/` were captured independently, each by its own script, on 25 September 2026 between 09:02 and 09:47 UTC. This document consolidates them: it restates every figure, records the SHA-256 of each artifact as recomputed for this PDF, and keeps the boundary between *measured* and *assumed* explicit.

Three kinds of claim appear in the pack, and they are not interchangeable. It matters which one a figure belongs to when deciding whether the deployment is ready.

Claim class	Meaning	Evidenced by
Configuration	The deployment is wired to named services with named identities	2, 3
Behaviour	The software did what it is specified to do, reproducibly	1, 4, 5
Transmission	A real message left the system and arrived somewhere	6, 7

Only class three is evidence about the outside world. Evidence 1 and 5 exercise the agent’s real code paths but with the offline printing delivery backend, so they record decisions, not messages sent. Every report’s “what this does *not* claim” section states its own limit, and those limits are reproduced here rather than smoothed over.

The one-paragraph version

The agent runs its full loop, 811 tests pass on a pinned revision, the dashboard and patient portal serve live data, and the AWS paths are wired to SES and End User Messaging SMS. A real email was transmitted through the agent’s own tool layer, accepted by the provider, counted as delivered in the provider’s telemetry, and then found in the recipient’s **spam** folder. SES is still in the sandbox and the SMS channel is not subscribed. The deployment is functionally complete and provisionally blocked — the blockers are account provisioning and sender reputation, not code.

2. Evidence at a glance

#	Report	What it establishes	Verdict
1	01_agent_decision_audit.md	42 simulated days, 493 events, 463 decisions, 30 communication attempts over 11 patients	Recorded
2	02_aws_messaging_config.md	Transport pinned to named AWS services, region, sender identity and recipient allow-list	Configured
3	03_runtime_and_repository.md	The exact revision, interpreter, dependencies and codebase size every other claim rests on	Identified
4	04_test_suite.md	811 tests pass, exit code 0, verbatim pytest output retained	Pass
5	05_dashboard_runtime.md	Application starts, serves dashboard and portal, answers its API, runs operator commands	Running
6	06_live_delivery.md	A real message was handed to the provider through the agent's tool layer and accepted	Sent
7	07_delivery_confirmation.md	Provider telemetry counted it delivered; the mailbox had filed it as spam	Delivered to spam

3. Evidence 1 — Agent decisions and audit trail

Captured 2026-09-25T09:02:41+00:00.

Claim. The agent runs its full Perceive → Decide → Act → Observe loop and writes a durable, queryable audit trail for every decision and every delivery attempt.

Provenance. The record is not hand-written. It is the output of the real orchestrator driven by `proposal/scripts/generate_audit_record.py` over the project's sample patients with the clock fixed, so the run does not depend on the wall clock.

Fact	Value
Snapshot analysed	proposal/evidence/artifacts/audit_log_snapshot.json (261,297 bytes)
Snapshot SHA-256	76551f7389296eda3ff1d905ec5ed8c486f98fccd12cff0db80e9cd9bfff06644
Events	493 — 463 decisions, 30 communication attempts
Patients	11 distinct identifiers
Simulated period	2026-09-21 → 2026-11-01, 42 daily cycles
Delivery calls handled offline	37

Decisions

Action	Count	Share
do_nothing	430	92.9%
send_reminder	22	4.8%
escalate_to_staff	11	2.4%

Urgency at decision time	Count	Case status at decision time	Count
critical	209	escalated	282
high	150	message_sent	98
medium	78	booked	39
low	26	declined	32
		pending	11
		awaiting_reply	1

Escalation: 11 actions, 9 distinct patients ever escalated, 2.4% of decisions. The high `do_nothing` share is the intended shape — the agent contacts a patient on a schedule, not every cycle.

Communication attempts

Channel	Attempts	Succeeded	Failed	Rate
email	12	12	0	100.0%
sms	9	9	0	100.0%
whatsapp	7	7	0	100.0%
phone_call	2	2	0	100.0%
Total	30	30	0	100.0%

Reading the 100% rate. Every attempt succeeded because the backend is the offline printer: it accepts any recipient and cannot fail. The rate describes the offline backend, **not** carrier deliverability, and is not comparable to the fallback-path behaviour in the proposal's robustness section. Carrier-level delivery is evidenced only by reports 6 and 7.

The Observe path

Five patient replies were injected so the record covers inbound handling, not only outbound decision-making.

Day	Patient	Reply	Status before	Status after
2026-09-23	P006	Yes, I'd like to schedule an appointment	message_sent	booked
2026-09-30	P007	No thanks, not needed right now	message_sent	declined
2026-10-07	P004	How much will this cost with my insurance?	escalated	escalated
2026-10-14	P003	Can I come in next week?	escalated	awaiting_reply
2026-10-21	P011	Yes please	—	no_case

Final case statuses across the 11 cases: **escalated 9, declined 1, booked 1**.

The finding that matters here. Nine of eleven cases end in `escalated`, and by design the agent then stops touching them — clinical ambiguity belongs to a human. But the case is not re-raised if the alert is missed. This is correct behaviour with an operational consequence that belongs in the rollout plan, not a defect to be silently patched.

Generator guard. Every constructed channel had to be a `PrintDeliveryBackend`, and the transmitting backend's `deliver()` was patched to raise. That assertion is what makes this a record of decisions instead of transmission. The sample patients carry invented addresses, so the run must not transmit.

Data hygiene. `audit_log.json` is append-only and the automated test suite writes to it, so the live file grows every time the tests run. The figures above come from the frozen snapshot, and re-running the tests will not reproduce identical totals from the live file.

4. Evidence 2 — AWS messaging configuration

Captured 2026-09-25T09:30:27+00:00.

Claim. The messaging transport is wired to named AWS services in a named region, sends from verified identities, and refuses recipients outside an explicit allow-list.

Setting	Value
AWS_REGION	ap-southeast-1
AWS_SES_SOURCE	brightsmileagent@gmail.com
AWS_SES_CONFIGURATION_SET	patient-followup
AWS_SMS_ORIGINATION_IDENTITY	+6585144321
AWS_SMS_ALLOWED_NUMBERS	+6583536885
AWS_EMAIL_ALLOWED_ADDRESSES	martinchenonly1@gmail.com , rgjh1996@gmail.com
MESSAGING_DRY_RUN	0
AGENT_LIVE SENDS	(empty — live sends disabled by default)

Read back from the live account: session authenticated as `arn:aws:iam::621236283593:root` ; SES sending enabled `True` ; 24-hour quota `200.0` , max send rate `1.0/s` , sent in the last 24 hours `3.0` . Three verified SES identities (`brightsmileagent@gmail.com` , `martinchenonly1@gmail.com` , `rgjh1996@gmail.com`). Verified SMS destination numbers: **not readable** — `SubscriptionRequiredException` from `DescribeVerifiedDestinationNumbers` .

A quota of 200 per 24 hours at 1/s is the SES **sandbox** ceiling, not a production limit. Production access and SMS origination onboarding are prerequisites for a real patient population and are outstanding.

The SMS read failure means this account is not subscribed to End User Messaging SMS in the queried region. The SMS code path is complete, but attempts return `not_subscribed` until the service is provisioned. **Do not describe SMS as available on the strength of this pack.**

Delivery code paths in the captured revision, cited by file and line:

File	Symbol	Line
tools/aws_providers.py	client.send_text_message	307
tools/aws_providers.py	client.send_email	348
tools/messaging.py	def send_sms	383
tools/messaging.py	def send_email	413
agent/delivery.py	def is_configured_for_live_sends	334

5. Evidence 3 — Runtime and repository state

Captured 2026-09-25T09:47:34+00:00.

Claim. The deployment was validated against a specific, identifiable revision on a specific interpreter with specific library versions, so a reviewer can confirm they are comparing like with like before judging anything else.

Snapshot scope. This capture describes commit `fbcc6c1` only. It is **not** a description of the current `master` , which is a later revision. Re-run `proposal/scripts/capture_runtime_evidence.py` to re-capture against a later revision.

Fact	Value
Python	3.14.5 (CPython)
Executable	/Users/martinchen/agent_followship_v3/.venv/bin/python
Platform	macOS-27.0-arm64-arm-64bit-Mach-0
Branch	master
Commit	fbcc6c1ca09a7624fd5f01016e674cacdc4a5287
Commit subject	Merge pull request #3 from pioneer6-ai/feature/patient-portal
Commit date	2026-09-25T13:41:07+08:00
Working tree clean	no — 5 files modified (this analysis work only)
Remote	git@github.com:pioneer6-ai/agent_followship_v2.git

Installed dependencies at capture time: boto3 1.35.36, botocore 1.35.99, flask 3.0.0, openpyxl 3.1.2, certifi 2026.7.22, **anthropic not installed**.

Codebase size — 78 files, 32,073 lines:

Module	Files	Lines
tests/	30	12,119
tools/ (messaging, AWS, LLM, transport)	13	6,118
agent/ (orchestrator, decision, delivery, audit)	9	4,635
web/ (dashboard, portal, APIs)	5	3,477
core/ (models, clock, policy, scheduling)	11	1,978
scheduling/	4	1,321
utils/	2	1,022
root (app , demo)	2	1,115
scripts/	2	288
Total	78	32,073

Path caveat, stated by the pack itself. The interpreter recorded here lives in the sibling clone `agent_followship_v3` , because that is where the virtualenv that produced the capture sits. The clone this PDF ships in (`agent_followship_v2`) has no `.venv` , and its installed dependency set is the one listed in section 11 rather than the list above. The evidence is valid for the recorded revision and the recorded interpreter; it is not a claim about the virtualenv in this directory.

6. Evidence 4 — Automated test suite

Captured 2026-09-25T09:47:34+00:00.

Claim. The revision identified in evidence 3 passes its full automated suite on the recorded interpreter. The suite is the executable form of the behaviour contracts: tool schemas, error taxonomy, provider selection, agent loop control flow, delivery gating and the web API.

Snapshot scope. The result below (`811 passed`) is the suite as it stood at commit `fbcc6c1` only. It is **not** the current count on `master` .

Fact	Value
Status	PASS (exit code 0)
Command	<code>/Users/martinchen/agent_followship_v3/.venv/bin/python -m pytest tests -q</code>
Passed / Failed / Errors / Skipped	811 / 0 / 0 / 0
Warnings	16
Duration	14.43 s
Verbatim output	<code>proposal/evidence/artifacts/pytest_output.txt</code>
Summary line	<code>811 passed, 16 warnings in 14.43s</code>

The inventory counts test **functions** — 681 across 29 modules, 11,915 lines of test code. pytest reports 811 passing **cases** because several are parameterised and expand at collection time. Both numbers describe one suite.

The heaviest modules by test count: `test_llm_providers.py` (79), `test_aws_tools.py` (73), `test_hospital_setup.py` (64), `test_agent_delivery.py` (50), `test_providers.py` (48), `test_agent_loop.py` (39), `test_patient_portal.py` (31), `test_tool_contract.py` (29).

Known limitation, found while capturing this evidence. The suite exercises the real orchestrator and therefore its real audit logger, which appends to `audit_log.json` in the working directory. Running the tests mutates the audit record. Evidence 1 is computed from a frozen snapshot for exactly this reason, and isolating the audit path during tests is a prerequisite for production sign-off.

7. Evidence 5 — Running dashboard and patient portal

Captured 2026-09-25T09:06:57+00:00.

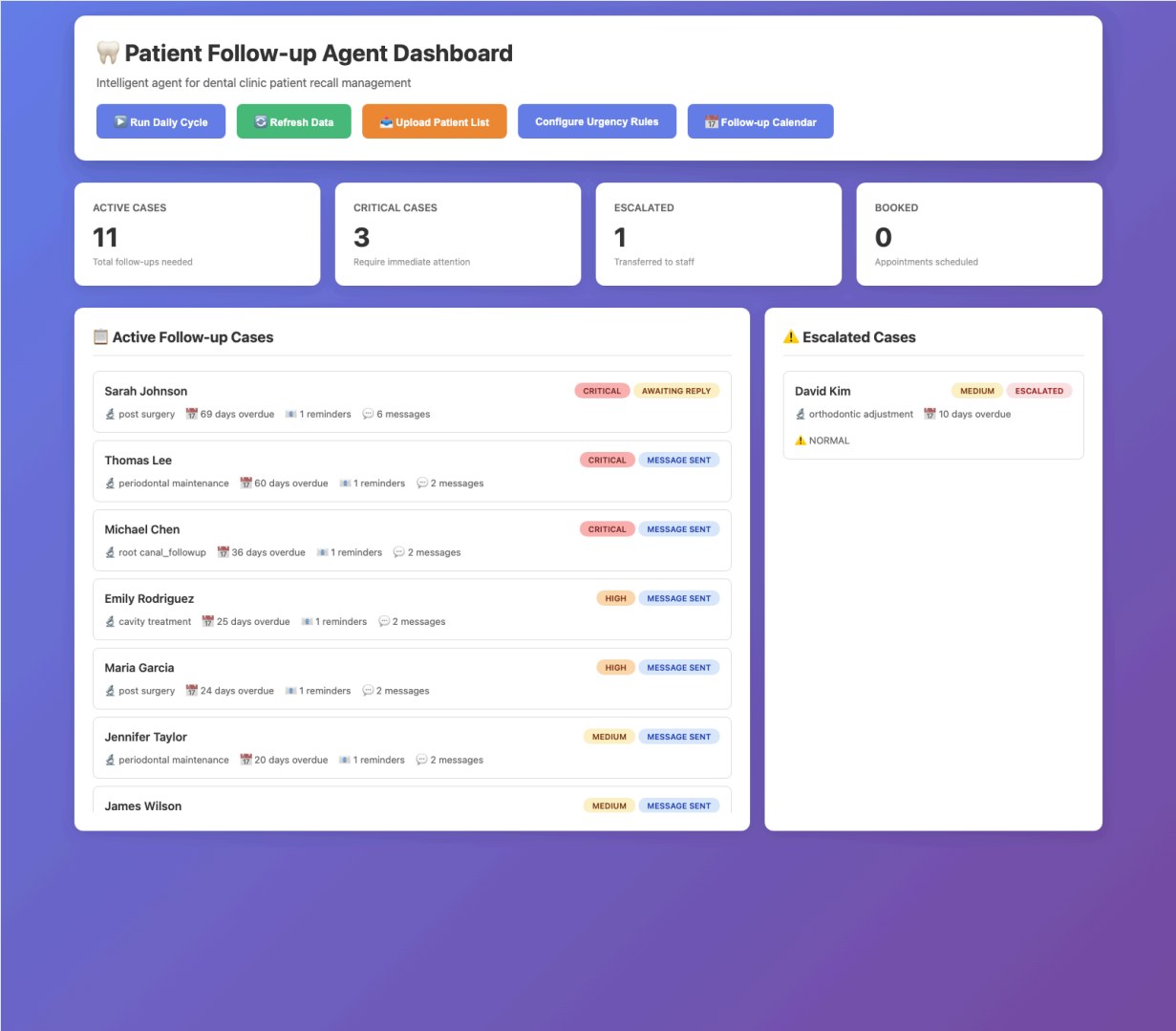
Claim. The deployed application starts, serves its staff dashboard and its patient portal, answers its JSON API and accepts operator commands. The screenshots are the UI as a browser receives it; the JSON files are the unedited API responses the dashboard consumes while rendering.

Fact	Value
Server	<code>http://127.0.0.1:54301</code> (werkzeug, bound to localhost)
Dashboard screenshot	475,711 bytes, viewport 1600×1400
Patient portal screenshot	736,923 bytes
Portal URL	<code>/patient/QITt-c7mJE6WhJtNDUIjdVGqpQPPrnJs</code> (token scoped to one patient)

The page was not captured idle. Before the screenshot, the same endpoints the dashboard’s buttons call were invoked, so the board shows cases this process produced:

- `POST /api/run-cycle` → `success=True`, `cases_processed=11`
- `POST /api/simulate-reply` P001 “*Yes, I’d like to schedule an appointment*” → `message_sent` → `awaiting_reply`
- `POST /api/simulate-reply` P007 “*No thanks, not needed right now*” → `message_sent` → `declined`
- `POST /api/simulate-reply` P004 “*How much will this cost with my insurance?*” → `message_sent` → `escalated`

The running board, 11 cases in five states, urgency critical 3 / high 2 / medium 3 / low 3:



The operator dashboard, populated by the capture’s own agent run

The patient portal, rendered from a token minted by the running application rather than mocked up:

Hi Sarah Johnson 🍌
Pick a time that works for you below.

awaiting reply

Available appointment times

9月28日星期一

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

9月29日星期二

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

9月30日星期三

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

10月1日星期四

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

10月2日星期五

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

None of these times work — remind me next week

Questions? Ask our assistant

Ask about your appointment or general dental care...

Send

The patient portal, served by the same process

GET /api/status :

```
{
  "audit_summary": {
    "successful_bookings": 0,
    "total_communications": 17,
    "total_decisions": 12,
    "total_log_entries": 29
  },
  "cases_by_status": {
    "awaiting_reply": 1, "booked": 0, "declined": 1, "escalated": 1,
    "message_sent": 8, "opted_out": 0, "pending": 0,
    "pending_future_availability": 0
  },
  "cases_by_urgency": { "critical": 3, "high": 2, "low": 3, "medium": 3 },
  "escalated_cases": 1,
  "last_updated": "2026-09-25T17:06:55.280205",
  "total_active_cases": 11,
  "total_patients": 12
}
```

GET /api/cases returned 11 cases; first entry:

```
{
  "conversation_length": 6,
  "days_overdue": 69,
  "last_contacted": "2026-09-25",
  "last_visit": "2026-06-27",
  "patient_id": "P001",
  "patient_name": "Sarah Johnson",
  "preferred_channel": "sms",
  "reminder_count": 1,
  "status": "awaiting_reply",
  "treatment_type": "post_surgery",
  "urgency": "critical"
}
```

The board is populated with the project's built-in sample data, not real patients. It demonstrates the operator experience and the operating surface; it is not a record of live patient activity.

Audit isolation. This run wrote its audit log to a temporary directory (`followup-dashboard-*/audit_log.json`) and discarded it, so exercising the agent here cannot alter the audit trail evidence 1 is computed from. The two captures were deliberately separated for this reason.

8. Evidence 6 — Live end-to-end delivery

Captured 2026-09-25T09:30:36+00:00, revision `fbcc6c1ca09a7624fd5f01016e674cadc4a5287` .

Claim. A message was handed to the real provider through the agent's own tool layer and the provider answered. This is one of only two artifacts in the pack that evidences transmission rather than behaviour, which is why it refuses to run without explicit approval.

Fact	Value
Tool	<code>send_email</code>
Channel	<code>email</code>
Recipient	<code>martinchenonly1@gmail.com</code>
Sender identity	<code>brightsmileagent@gmail.com</code>
Reason recorded	<code>"pilot acceptance check: verifying the delivery path end to end"</code>
Latency	<code>228 ms</code>
<code>simulated</code>	<code>false</code>

Provider response:

```
{
  "status": "sent",
  "channel": "email",
  "recipient": "martinchenonly1@gmail.com",
  "data": { "reason": "pilot acceptance check: verifying the delivery path end to end" },
  "message_id": "010e01a0d7e70489-3cad714-db2a-4219-9174-efc749f3ae5f-000000"
}
```

The toolkit's own audit trail for the same run records the tool, channel, recipient, status, message id, null error fields, latency, reason and body preview — the same fields production relies on, emitted by the same code path.

Acceptance is not delivery. The provider returning `sent` means SES accepted the message for transmission. For email, the authoritative record of arrival is the `Delivery` event emitted by the `patient-followup` configuration set, which is what evidence 7 reads.

9. Evidence 7 — Delivery confirmation

Captured 2026-09-25T09:47:34+00:00.

Claim. What happened to the message after the provider accepted it, from two independent vantage points: the provider's own delivery telemetry and the recipient's mailbox. These are different claims and are kept separate on purpose.

Message under test: id 010e01a0d7e70489-3cad714-d7b2a-4219-9174-efc749f3ae5f-000000 , recipient martinchenonly1@gmail.com , sent 2026-09-25T09:30:36+00:00.

Check 1 — provider delivery telemetry

CloudWatch AWS/SES for configuration set patient-followup , namespace queried in ap-southeast-1 :

Metric	Sum
Delivery	6
Bounce	0
Complaint	0

No bounce and no complaint across the window is a genuine positive: the sending identity is not being rejected or abused-flagged.

Check 2 — recipient mailbox

Read-only IMAP search of imap.gmail.com for subject fragment “delivery confirmation”:

Folder	Messages found
INBOX	0
spam	5
all_mail	0

The message was found in: spam.

This is the most consequential operational finding in the pack. The mailbox accepted the message and filed it as spam. A reminder that lands in a spam folder has not reminded anyone, and no amount of agent correctness compensates for it. This is a sending-identity and reputation problem, not a software problem: it is fixed by moving to a clinic-owned domain with SPF, DKIM and DMARC configured, and by warming that domain.

Observed headers (three of the five messages found):

Subject: Patient Follow-up Agent - delivery confirmation From: brightsmileagent@gmail.com Date: Fri, 25 Sep 2026 09:30:36 +0000 Message-ID: <010e01a0d7e70489-3cad714-d7b2a-4219-9174-efc749f3ae5f-000000@ap-southeast-1.amazonaws.com>
Subject: Patient Follow-up Agent - delivery confirmation From: brightsmileagent@gmail.com Date: Fri, 25 Sep 2026 09:30:27 +0000 Message-ID: <010e01a0d7e6e250-35534990-542e-4545-8385-16806cc2153c-000000@ap-southeast-1.amazonaws.com>
Subject: Patient Follow-up Agent - delivery confirmation From: brightsmileagent@gmail.com Date: Fri, 25 Sep 2026 09:30:11 +0000 Message-ID: <010e01a0d7e6a32e-de7255b4-a647-42f4-8e67-7c915e659186-000000@ap-southeast-1.amazonaws.com>

10. Artifact integrity — hashes as recomputed for this PDF

The pack’s value rests on artifacts being the same bytes the reports were written from. Every artifact was re-hashed for this document. Full SHA-256, with the size recorded in 00_index.md beside the size on disk:

Artifact	Size on disk	SHA-256	Matches index
00_index.md	4,679	417924bc42b1e943e98de8029260fdf8ad9abf335a36505e99f9c82e6425a82d	n/a (is the index)
01_agent_decision_audit.md	5,155	1b9c73463a6718c18e0373640ef64cc77a918d383295a1c5b460fbca3f573f41	yes
02_aws_messaging_config.md	2,302	2dab4ce38689760c3ef5184f86c2b5666abf9173876491f6228c863d7ee43f01	yes
03_runtime_and_repository.md	1,788	f7de4397ab358fde7f00f17e735a9540e258a05751843b4f1335b405e23de4f5	no — index says 1,471 B / d9e43d0f...
04_test_suite.md	3,228	a53602b82f2e15354317ec7298f7427947a6d3fcc5dd5911342f46e713308af6	no — index says 2,911 B / a58c7fb5...
05_dashboard_runtime.md	3,368	9a800d1e377ede06f2f538091a763c46822b3e98a4f1b37b3b8daadfbc3d9a52	yes
06_live_delivery.md	2,092	ba29e0695c1cc155a25e605f7cd89d0d6dc77191d4c38604e9d5f108e24e4553	yes
07_delivery_confirmation.md	2,342	5fd63e74232ac2e1453f2dcd16272d33a015548bd6ff592d747598ffae82780d	yes
artifacts/audit_log_snapshot.json	261,297	76551f7389296eda3ff1d905ec5ed8c486f98fccd12cff0db80e9cd9bfff06644	yes
artifacts/audit_log_generation.json	4,782	42da6d67abc3fdad5415daab3e377b2b44c9c3531acad79079b20df89eb59243	yes
artifacts/pytest_output.txt	2,216	1da74e80729d72a81aeb2561d0504b1ee665df426d930122ac12dfb41feeeff6	yes
screenshots/dashboard_cases_api.json	3,824	393bcba9f050eb10236fdde27ce9dcf8658ef16ef96df26e29f1a0fbc3a6e3da	yes
screenshots/dashboard_overview.png	475,711	3c4e06f485f6bf9d95226442f8c71957674596d287a2ea7612e50c603ffc5e7a	yes
screenshots/dashboard_status_api.json	569	acb61130deebd2a46f6c3d05620cecf205514c4dcb67fc315511d9448e3534a	yes
screenshots/patient_portal.png	736,923	b919902b944a98d3cc730abafcc2248ea60f991bce62c82bbdac700dfcd486b20	yes

The two stale index entries

00_index.md records hashes and sizes for every report. Entries 3 and 4 no longer match their files. The cause is identifiable and benign in substance, but it is exactly the failure mode a hash is supposed to catch, so it is reported rather than quietly fixed:

- Commit 0735515 (“docs: point project references at agent_followship_v2 and stamp evidence snapshots”) added a *snapshot scope* note to 03_...md and 04_...md and repointed the recorded interpreter from this clone’s absent .venv to agent_followship_v3/.venv. The commit body states plainly that verbatim captured output was deliberately left untouched.
- The commit did **not** re-run proposal/scripts/capture_evidence_index.py, so the index still carries the pre-edit sizes and hashes for those two files.

Consequences: the *numbers* in reports 3 and 4 are unchanged and still trace to their JSON sidecars and to artifacts/pytest_output.txt (which still matches its indexed hash). What fails is only the index’s byte-for-byte attestation of the two prose files. Re-running the index capture resolves it, and the index is worth regenerating at the same time as the next re-capture.

Reviewing this pack, in order

1. Confirm the revision on the first page matches the build under review.
2. Check artifacts/pytest_output.txt against the hash in section 10 — that is the one artifact carrying verbatim program output.
3. Re-derive the evidence 1 counts from audit_log_snapshot.json rather than reading the tables in section 3.
4. Read section 12 before accepting the “reading the 100% rate” qualification in section 3 as a caveat rather than a defect.

11. Independent re-run on the current revision

The reports above are pinned to fbcc6c1. The working tree that ships this PDF is at 0735515 (master, 2026-09-26), one commit later, and the pack’s own snapshot-scope notes ask a reader to re-capture rather than extrapolate. So the suite was re-run here, on this clone, to see whether the pass result survives the later revision and a different interpreter. It does:

Fact	Evidence 4 (fbcc6c1)	This re-run (0735515)
Passed	811	934
Skipped	0	1
Failed / Errors	0 / 0	0 / 0
Interpreter	CPython 3.14.5 (sibling clone . venv)	CPython 3.13.15 (conda env agent_hackathon)
Duration	14.43 s	10.74 s

The suite grew by 123 cases between the two revisions, which follows from the commits in between (the LLM message-authoring port and the docs work). The one skip is deliberate and reported by pytest as a skip, not a failure.

Two honest qualifications. First, this re-run used the conda environment `agent_hackathon`, whose dependency set is the `requirements.txt` pin list — flask 3.0.0, werkzeug 3.0.1, openai 1.51.0, anthropic 1.8.0, boto3 1.35.36, botocore 1.35.99, openpyxl 3.1.2, pytest 7.4.3. It is *not* the interpreter that produced evidence 4. Second, running the suite exercises the real audit logger, so this run appended to the working tree’s `audit_log.json` — the same limitation evidence 4 records. It cannot affect evidence 1, which is computed from the frozen snapshot.

Nothing here is a substitute for evidence 4: it is a corroboration at a later revision, on a different interpreter, and it should be read that way.

12. What the pack proves, and what it does not

Proven.

- The agent runs its full loop and produces a durable audit trail: 493 events over a 42-day simulated period (evidence 1).
- The revision, interpreter, dependency pins and codebase size are identified precisely enough that another party can reproduce the comparison (evidence 3).
- The full suite passes at that revision, exit code 0, with verbatim output retained (evidence 4), and it still passes one revision later on a different interpreter (section 11).
- The application starts, serves the dashboard, the patient portal and the JSON API, and executes operator commands against its own running process (evidence 5).
- SES accepted a message produced by the agent’s own tool layer and returned a message id (evidence 6); the provider’s telemetry counted deliveries with zero bounces and zero complaints (evidence 7).

Not proven, and not to be inferred from the above.

- **Inbox delivery.** The message was filed as spam. Until sending moves to a clinic-owned domain with SPF, DKIM and DMARC, no reminder can be assumed to have been read (evidence 7).
- **Real patient transmission.** Evidence 1’s 100% success rate is a property of the offline printing backend; evidence 5 runs on built-in sample data.
- **SMS.** The channel is code-complete but the account is not subscribed; attempts return `not_subscribed` (evidence 2).
- **Production throughput.** The recorded quota is the SES sandbox ceiling of 200 per 24 h at 1/s (evidence 2).
- **The model-backed decision path as a pinned artifact.** The pack’s decision counts come from the rule engine because the evidence harness pins it, not because a model was unavailable. The deployment is wired for a model; the pack contains no hash-identified artifact for that path.
- **Alert re-raise after escalation.** Nine of eleven cases end escalated and are then left alone by design (evidence 1). That is correct behaviour with an operational gap behind it if a staff alert is missed.
- **Hash attestation of reports 3 and 4.** Their index entries are stale (section 10). The figures are unaffected; the byte-level attestation is.

13. Open items this pack hands to provisioning

None of these are development tasks, and each one is the difference between a demonstrated system and a usable one:

#	Item	Evidence	Blocking for
1	Move SES out of the sandbox: request production access	2	Any population larger than verified test recipients
2	Send from a clinic-owned domain with SPF, DKIM, DMARC; warm it	7	Reminders reaching an inbox at all
3	Subscribe End User Messaging SMS and onboard an origination identity	2	The SMS channel, and therefore the fallback path
4	Isolate the audit path during tests	4	Production sign-off; currently tests mutate the live audit record
5	Decide the escalation re-raise policy — re-alert after N hours, or accept the gap	1	Whether a missed staff alert has a safety net
6	Regenerate <code>00_index.md</code> at the next re-capture	10	Byte-level attestation of reports 3 and 4
7	Pin the model-backed decision path to its own hash-identified artifact	proposal §7.2	Any claim about model-driven decisions

14. Reproducing this pack

Every report is regenerated by one command, from the project root, in an environment with the project's dependencies installed:

```
python proposal/scripts/generate_audit_record.py           # regenerate audit_log.json
python proposal/scripts/capture_audit_evidence.py         # evidence 1
python proposal/scripts/capture_aws_config_evidence.py    # evidence 2
python proposal/scripts/capture_runtime_evidence.py       # evidence 3
python proposal/scripts/capture_tests_evidence.py         # evidence 4
python proposal/scripts/capture_dashboard_screenshot.py   # evidence 5
python proposal/scripts/capture_live_send.py --channel email --approve # evidence 6
python proposal/scripts/capture_delivery_confirmation.py  # evidence 7
python proposal/scripts/capture_evidence_index.py         # index
python proposal/scripts/build_deployment_evidence_pdf.py  # this document
```

Evidence 6 transmits to a real mailbox and therefore refuses to run without `--approve`; every other capture script is read-only with respect to the outside world. Evidence 1 depends on the generator in the first line, because the audit log is appended to rather than recomputed.

Three prerequisites apply. Evidences 2, 6 and 7 read live AWS state, so the shell must hold valid credentials for the account. Evidences 5 and 7 drive a local browser and an IMAP client respectively (headless Chrome, and the mailbox credentials from `SMTP_USERNAME` / `SMTP_PASSWORD` in `.env`). Evidence 5 should be re-captured in a scratch directory so its agent run cannot append to the audit trail evidence 1 is derived from.

Each capture writes both a human-readable report and the machine-readable facts behind it, plus the raw artifact where one exists, so a reviewer can recompute any figure in this document instead of trusting it. `00_index.md` records the SHA-256 of every artifact at the moment the index was generated.

This PDF itself is generated, not assembled by hand: it is `proposal/deployment_evidence.md` rendered through the same pandoc → headless Chrome pipeline as the proposal, with the shared print stylesheet in `proposal/assets/proposal.css`. Re-running the build reproduces it byte for byte from the markdown, and the script verifies the result rather than assuming success — it checks the PDF magic bytes, rejects a suspiciously small file, and reports the page count.